Travaux Pratiques : Affichage de texte défilant sur matrice de leds multiplexée avec FPGA

Superviseur : Peace ADIKPETO

Contexte

On veut refaire l'enseigne de l'epac en mode défilant pour afficher « **EPAC** ». On te demande donc de réaliser le bandeau sur une matrice de leds multiplixée 8x12 en utilisant le FPGA EP4CE22F17 de la classe CYCLONE IV E.

Objectif : Montrer sa maitrise du langage VHDL, des logiciels ***Quartus & ModelSim***

Ressources

- **Matérielles**
  - ✓ Altera Cyclone® IV EP4CE22F17C6N FPGA (**Terasic DE0-NANO Board**)
  - ✓ USB Type mini-AB port (5V)
  - ✓ EPFL/LAP ExtDE0_LedSW
- **Logicielles**
  - ✓ Quartus Prime Lite Edition
  - ✓ ModelSim Altera

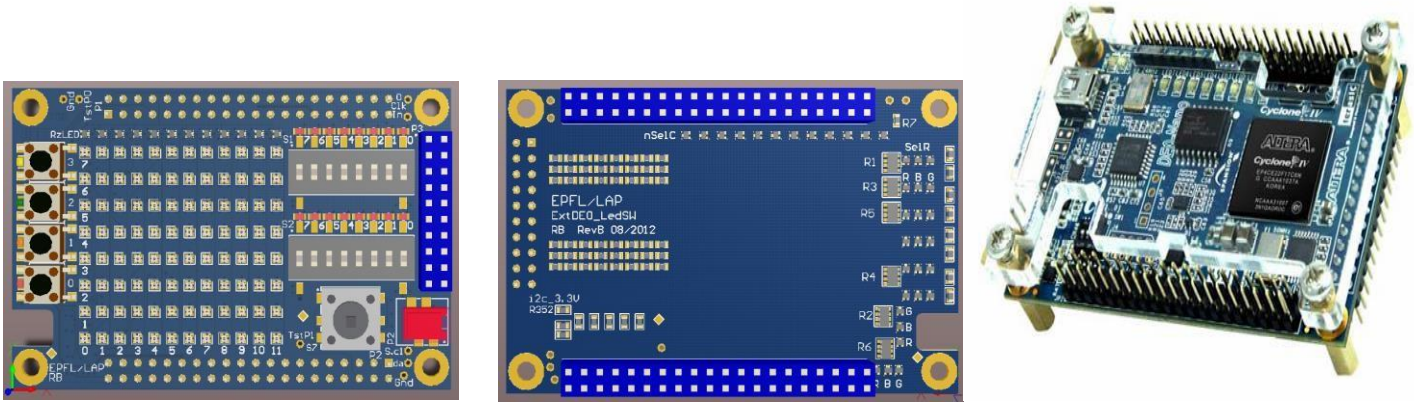


Figure 1 : Matériels A

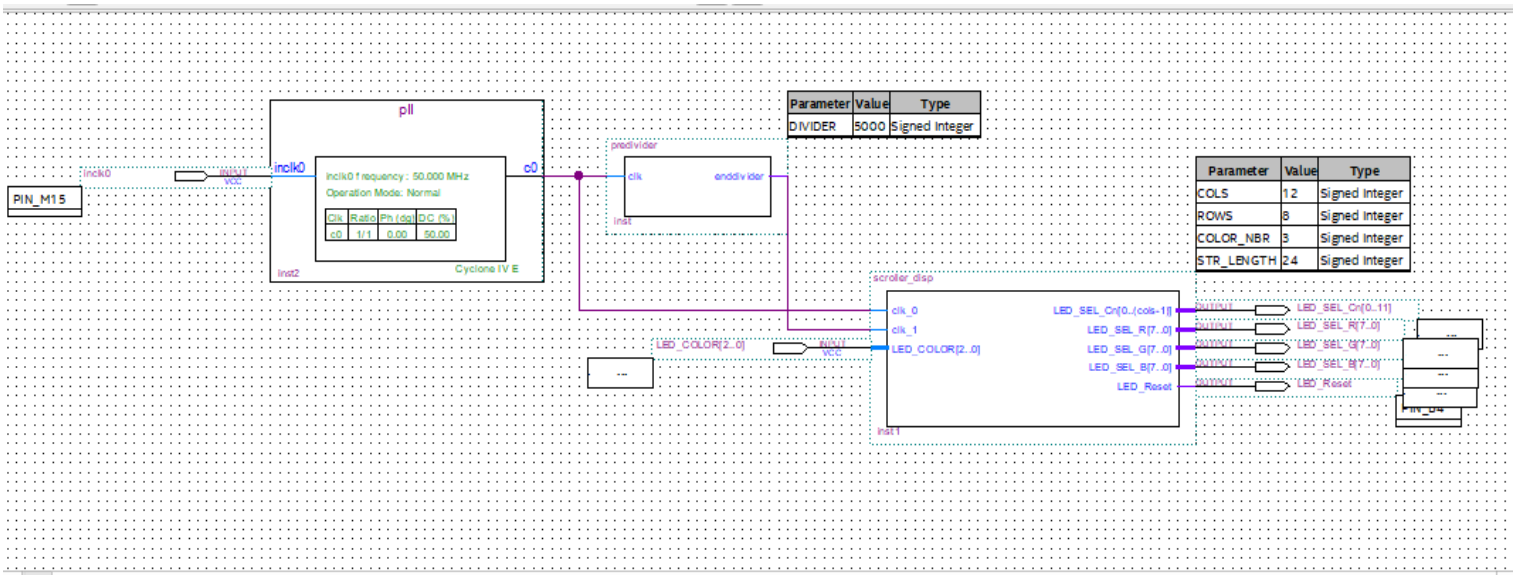
savoir !

- **Matrice de LED simple (non multiplexée)** : c'est un écran composé de diodes électroluminescentes disposées en lignes et en colonnes. Par défaut lorsqu'une ligne est active, toutes les leds de cette ligne le sont, de même pour une colonne (les leds sont à cathodes ou à anode commune).

- **Le multiplexage** : C'est une technique électronique qui consiste à allumer les LED très rapidement (une par une ou par ligne) à tour de rôle, créant une illusion visuelle de la lumière continue grâce à la persistance rétinienne.

- **La persistance rétinienne** : C'est la capacité de l'œil humain et du cerveau à conserver en mémoire une image lumineuse pendant une fraction de seconde après sa disparition. Cela crée une illusion d'optique.

- **Le défilement de texte ou scrolling** : C'est une astuce qui permet d'afficher un message plus long que la taille de la matrice. Son principe de base est celle de la fenêtre glissante, le texte complet est stocké sur une bande virtuelle plus longue que la matrice. Ainsi, on décale la bande d'une colonne vers la gauche à intervalle régulier. **Schéma bloc sous quartus**



Code VHDL de chaque Bloc

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

package text_pkg is
    type pixel_data is array (0 to 23) of bit_vector(7 downto 0);
    type cor is array (2 downto 0) of bit_vector(7 downto 0);

    constant labels : pixel_data := (
        X"FE", X"92", X"92", X"92",  -- Letter E
        X"00", X"00",                 -- Space
        X"FE", X"90", X"90", X"F0",  -- Letter P
        X"00", X"00",                 -- Space
        X"FE", X"90", X"90", X"FE",  -- Letter A
        X"00", X"00",                 -- Space
        X"FE", X"82", X"82", X"82",  -- Letter C
        X"00", X"00"                  -- Space
    );
end package text_pkg;
```

- ✓ **Prédiviseur**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity predivider is
    generic (
        DIVIDER : integer := 50000000
    );
    port (
        clk        : in  std_logic;
        reset      : in  std_logic;
        enddivider : out std_logic
    );
end entity predivider;

architecture bhv of predivider is
    signal cnt : natural range 0 to DIVIDER := 0;
begin
    process(clk, reset)
    begin
        if reset = '1' then
            cnt <= 0;
            enddivider <= '0';
        elsif rising_edge(clk) then
            if cnt >= DIVIDER - 1 then
                cnt <= 0;
        elsif rising_edge(clk) then
            if cnt >= DIVIDER - 1 then
                cnt <= 0;
                enddivider <= '1';
            else
                cnt <= cnt + 1;
                enddivider <= '0';
            end if;
        end if;
    end process;
end architecture bhv;
```

- ✓ **Scroller_disp**

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use WORK.text_pkg.ALL;

entity scroller_disp is
    generic (
        COLS       : integer := 12;
        ROWS       : integer := 6;
        COLOR_NBR  : integer := 3;
        STR_LENGTH : integer := 24
    );
    port (
        clk_0       : in  std_logic;  -- pixel refresh clock
        clk_1       : in  std_logic;  -- scroll clock
        LED_COLOR   : in  std_logic_vector(2 downto 0);
        LED_SEL_Cn  : out bit_vector(0 to COLS-1);
        LED_SEL_R   : out bit_vector(7 downto 0);
        LED_SEL_G   : out bit_vector(7 downto 0);
        LED_SEL_B   : out bit_vector(7 downto 0);
        LED_Reset   : out bit
    );
end entity scroller_disp;

architecture rtl of scroller_disp is
    signal colm     : bit_vector(COLS-1 downto 0) := (others => '1');
    signal col      : cor := (others => X"00");
    signal nbr      : bit_vector(ROWS-1 downto 0) := (others => '0');
    signal buffer_0 : pixel_data := (others => X"00");
begin
    LED_SEL_Cn <= colm;
    LED_SEL_R  <= col(2);
    LED_SEL_G  <= col(1);
    LED_SEL_B  <= col(0);
    LED_Reset  <= '0';

    -- Column multiplexing / pixel refresh
    disp : process(clk_0)
        variable cnt_0, prev : natural range 0 to COLS-1 := 0;
    begin
        if rising_edge(clk_0) then
            nbr <= buffer_0(COLS-1 - cnt_0);
            colm(prev) <= '1';
            colm(cnt_0) <= '0';
            for i in 2 downto LED_COLOR'low loop
                if LED_COLOR(i) = '0' then
                    col(i) <= nbr;
                else
                    col(i) <= (others => '0');
                end if;
            end loop;
            prev := cnt_0;
```

```
            if cnt_0 = COLS-1 then
                cnt_0 := 0;
            else
                cnt_0 := cnt_0 + 1;
            end if;
        end if;
    end process;

    -- Scrolling shift register
    scroll : process(clk_1)
        variable cnt_1 : natural range 0 to STR_LENGTH-1 := 0;
    begin
        if rising_edge(clk_1) then
            for i in 0 to COLS-2 loop
                buffer_0(i+1) <= buffer_0(i);
            end loop;
            buffer_0(0) <= labels(STR_LENGTH - 1 - cnt_1);
            if cnt_1 = STR_LENGTH - 1 then
                cnt_1 := 0;
            else
                cnt_1 := cnt_1 + 1;
            end if;
        end if;
    end process;
end architecture rtl;
```

Pin planner

To	Value
inclk0	PIN_M15
LED_COLOR[2]	PIN_E9
LED_COLOR[1]	PIN_C9
LED_COLOR[0]	PIN_D9
LED_Reset	PIN_B4
LED_SEL_B[7]	PIN_T11
LED_SEL_B[6]	PIN_P11
LED_SEL_B[5]	PIN_P9
LED_SEL_B[4]	PIN_L16
LED_SEL_B[3]	PIN_L15

LED_SEL_B[2]	PIN_R14
LED_SEL_B[1]	PIN_P14
LED_SEL_B[0]	PIN_M10
LED_SEL_G[7]	PIN_T10
LED_SEL_G[6]	PIN_R10
LED_SEL_G[5]	PIN_N9
LED_SEL_G[4]	PIN_K16
LED_SEL_G[3]	PIN_P15
LED_SEL_G[2]	PIN_N16
LED_SEL_G[1]	PIN_L14
LED_SEL_G[0]	PIN_L13
LED_SEL_R[7]	PIN_R12
LED_SEL_R[6]	PIN_R11
LED_SEL_R[5]	PIN_N12
LED_SEL_R[4]	PIN_N11
LED_SEL_R[3]	PIN_R16
LED_SEL_R[2]	PIN_P16
LED_SEL_R[1]	PIN_N15
LED_SEL_R[0]	PIN_N14
LED_SELC_n[0]	PIN_B5
LED_SELC_n[1]	PIN_A4
LED_SELC_n[2]	PIN_D5
LED_SELC_n[3]	PIN_A5
LED_SELC_n[4]	PIN_A6
LED_SELC_n[5]	PIN_B6

LED_SELC_n[6]	PIN_D6
LED_SELC_n[7]	PIN_B7
LED_SELC_n[8]	PIN_C6
LED_SELC_n[9]	PIN_A7
LED_SELC_n[10]	PIN_E6
LED_SELC_n[11]	PIN_C8